ASSIGNMENT 3

Feature Engineering and Word Vectorization

Sr. No.	Name of Student	PRN	CO Mapped
1	Dhawse Spandan Yashwant	23070122085	CO4
2	Garvit Tyagi	23070122094	CO4

CA3 Project Title	RedactIQ: Explainable PII Detection and Redaction with Confidence-Scored Audit Trails

Aim

To convert text into numerical features using one-hot encoding, Bag of Words and TF-IDF, to train Naive Bayes and Logistic Regression classifiers that decide whether the context around a token carries personally identifiable information, and to learn dense word vectors with word2vec using both the CBOW and the skip-gram architectures.

The features learned here are used to justify the context-boosting rule of the confidence model built in Assignment 2 of the CA3 project RedactIQ.

Learning Outcome:

After completing this experiment, students will be able to:

- Implement Bag of Words and TF-IDF vectorization of text using scikit-learn and interpret the resulting document-term matrix.
- Train and compare Naive Bayes and Logistic Regression classifiers on the resulting feature representations.
- Train word2vec embeddings using the CBOW and skip-gram architectures and interpret the learned vector space.

Theory

A machine learning model requires numerical input, so text must first be converted into vectors. In one-hot encoding each word becomes a vector with a single 1 at its index in the vocabulary and 0 everywhere else. Such vectors are extremely sparse, their length equals the vocabulary size, and every pair of distinct words is equally dissimilar, so no notion of meaning is captured. This is the curse of dimensionality that motivates the representations below.

Bag of Words (BoW) represents a document by the counts of the words it contains, discarding word order. TF-IDF refines this by weighting each term as $w(t,d) = tf(t,d) \times \log(N / df(t))$, where tf is the count of the term in the document, df is the number of documents containing it and N is the total number of documents. The inverse document frequency suppresses terms that occur almost everywhere, and the logarithm damps the weight of extremely rare terms; the result is that common function words carry little weight while informative terms retain theirs.

Two classifiers are trained on these features. Multinomial Naive Bayes applies Bayes' theorem under the assumption that features are conditionally independent given the class. That assumption is clearly false for language, yet the model performs well on sparse text features and is very fast. Logistic Regression instead learns a weight for each feature and models the class probability through the

sigmoid function; it makes no independence assumption and usually performs better when features are correlated, as words in a context window are.

Word embeddings replace sparse counts with short dense vectors in which semantically related words lie close together. word2vec learns them in two ways: CBOW predicts a target word from its surrounding context and is faster, while skip-gram predicts the context from the target word and gives better vectors for rare words. Training the full softmax over the vocabulary is expensive, so negative sampling is used, in which the model distinguishes the true context word from a small number of randomly sampled words. Similarity between two learned vectors is measured by their cosine.

In this experiment the target token is replaced by the placeholder <TARGET> before vectorization. The classifier is therefore forced to learn from the surrounding words alone and never sees the entity string itself, so what it learns transfers to names and identifiers it has never encountered.

Algorithm

1. Import the required components from gensim and scikit-learn and load the spaCy model used for tokenization.
2. Generate a labelled corpus of sentences from a set of templates, recording the character span of every PII item so that the labels are exact.
3. For every token in every sentence, extract a symmetric window of three tokens on each side, replace the token itself by the placeholder <TARGET>, and label the window 1 if the token lies inside a PII span and 0 otherwise.
4. Shuffle the windows with a fixed seed and split them into a training set and a test set in the ratio three to one.
5. Fit a CountVectorizer and a TfidfVectorizer on the training windows and transform both sets, obtaining the Bag of Words and TF-IDF document-term matrices.
6. Display the terms with the lowest and the highest inverse document frequency to verify that common words are suppressed.
7. Train Multinomial Naive Bayes and Logistic Regression on each of the two representations and report accuracy, precision, recall and F1 on the test set.
8. Rank the vocabulary by the log-probability ratio of the Naive Bayes model to identify the context words most indicative of PII.
9. Tokenize the corpus for embedding training, discarding tokens containing digits so that the context vocabulary is learned rather than the identifiers themselves.
10. Train a CBOW model and a skip-gram model with negative sampling, then display the nearest neighbours and the pairwise cosine similarities of the trigger words.

Program

```
import random
import spacy
from gensim.models import Word2Vec
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_recall_fscore_support, accuracy_score

nlp = spacy.load("en_core_web_sm")
random.seed(42)

# ---------------- Step 1 : Build a labelled corpus ----------------
NAMES = ["Rohan Mehta", "Anita Desai", "Kabir Rao", "Farhan Qureshi",
         "Priya Sharma", "Neha Kulkarni", "Arjun Patel", "Sara Khan"]
ORGS = ["Infosys Limited", "Wipro Technologies", "Symbiosis Institute"]

def pan():
    return "".join(random.choices("ABCDEFGHIJKLMNOPQRSTUVWXYZ", k=5)) + \
           "".join(random.choices("0123456789", k=4)) + random.choice("ABCDEFGH")

def phone():
    return str(random.randint(6, 9)) + "".join(random.choices("0123456789", k=9))

def mail(n):
    return n.lower().replace(" ", ".") + "@sitpune.edu.in"
```

```
def build(parts):
    """parts = [(chunk, is_pii), ...] -> sentence text and its PII spans."""
    text, spans = "", []
    for chunk, pii in parts:
        if pii:
            spans.append((len(text), len(text) + len(chunk)))
        text += chunk
    return text, spans

TEMPLATES = [
    lambda n: build([("Please contact ", 0), (n, 1), (" at ", 0), (mail(n), 1),
                     (" or on ", 0), (phone(), 1), (" today.", 0)]),
    lambda n: build([("Employee ", 0), (n, 1), (" of ", 0),
                     (random.choice(ORGS), 1), (" has PAN ", 0), (pan(), 1),
                     (".", 0)]),
    lambda n: build([("Ticket raised by ", 0), (n, 1),
                     (". Registered mobile is ", 0), (phone(), 1), (".", 0)]),
    lambda n: build([("Kindly forward the offer letter to ", 0), (n, 1),
                     (" at ", 0), (mail(n), 1), (" before Friday.", 0)]),
    lambda n: build([("The applicant ", 0), (n, 1), (" submitted PAN ", 0),
                     (pan(), 1), (" for verification.", 0)]),
    lambda n: build([("Our records show that ", 0), (n, 1),
                     (" works with ", 0), (random.choice(ORGS), 1), (".", 0)]),
    lambda n: build([("The compliance review meeting is scheduled next week.", 0)]),
    lambda n: build([("All departments must submit the quarterly audit report.", 0)]),
    lambda n: build([("No personal identifiers are present in this document.", 0)]),
]
corpus = [random.choice(TEMPLATES)(random.choice(NAMES)) for _ in range(800)]
print("1. CORPUS :", len(corpus), "sentences")
print("   sample :", corpus[0][0])
```

```python
# ---------------- Step 2 : Context windows around each token ----------------
WIN = 3
def windows(text, spans):
    """Label each token 1 if it lies inside a PII span. The token itself is
    masked as <TARGET> so only the surrounding context is used as a feature."""
    doc, rows = nlp(text), []
    for i, tok in enumerate(doc):
        if tok.is_punct or tok.is_space:
            continue
        s, e = tok.idx, tok.idx + len(tok.text)
        y = int(any(a <= s and e <= b for a, b in spans))
        left = [t.text.lower() for t in doc[max(0, i - WIN):i]]
        right = [t.text.lower() for t in doc[i + 1:i + 1 + WIN]]
        rows.append((" ".join(left + ["<TARGET>"] + right), y))
    return rows

data = [r for t, s in corpus for r in windows(t, s)]
random.Random(42).shuffle(data)                # shuffle before splitting
split = int(0.75 * len(data))
Xtr_t, ytr = zip(*data[:split])
Xte_t, yte = zip(*data[split:])
print(f"\n2. CONTEXT WINDOWS : {len(data)} total, "
      f"{sum(ytr)} PII / {len(ytr)-sum(ytr)} non-PII in train")
print("   example (PII) :", next(w for w, y in data if y == 1))
```

```python
# ---------------- Step 3 : BoW and TF-IDF vectorization ----------------
bow = CountVectorizer(token_pattern=r"\S+", min_df=2)
tfidf = TfidfVectorizer(token_pattern=r"\S+", min_df=2)
Xb_tr, Xb_te = bow.fit_transform(Xtr_t), bow.transform(Xte_t)
Xt_tr, Xt_te = tfidf.fit_transform(Xtr_t), tfidf.transform(Xte_t)
print(f"\n3. VECTORIZATION")
print(f"   BoW    matrix {Xb_tr.shape}  vocabulary = {len(bow.vocabulary_)}")
print(f"   TF-IDF matrix {Xt_tr.shape}  w(t,d) = tf(t,d) x log(N / df(t))")

idf = sorted(zip(tfidf.get_feature_names_out(), tfidf.idf_), key=lambda x: x[1])
print("   lowest  IDF (common)    :", ", ".join(t for t, _ in idf[:5]))
print("   highest IDF (informative):", ", ".join(t for t, _ in idf[-3:]))
```

```python
# ---------------- Step 4 : Classification ----------------
print(f"\n4. PII-CONTEXT CLASSIFICATION")
print(f"   {'Features + model':<28}{'Accuracy':<11}{'Prec':<8}{'Recall':<9}F1")
for name, A, B, clf in [
        ("BoW    + MultinomialNB", Xb_tr, Xb_te, MultinomialNB()),
        ("TF-IDF + MultinomialNB", Xt_tr, Xt_te, MultinomialNB()),
        ("BoW    + LogisticReg",   Xb_tr, Xb_te, LogisticRegression(max_iter=1000)),
        ("TF-IDF + LogisticReg",   Xt_tr, Xt_te, LogisticRegression(max_iter=1000))]:
    pred = clf.fit(A, ytr).predict(B)
    P, R, F, _ = precision_recall_fscore_support(yte, pred, average="binary",
                                                 zero_division=0)
    print(f"   {name:<28}{accuracy_score(yte, pred):<11.3f}{P:<8.3f}{R:<9.3f}{F:.3f}")

nb = MultinomialNB().fit(Xb_tr, ytr)
ratio = nb.feature_log_prob_[1] - nb.feature_log_prob_[0]
top = sorted(zip(bow.get_feature_names_out(), ratio), key=lambda x: -x[1])
top = [t for t, _ in top if t.isalpha()][:8]
print("   context words most indicative of PII :")
print("     ", ", ".join(top))
```

```python
# ---------------- Step 5 : word2vec word vectors ----------------
sents = [[w.strip(".,").lower() for w in t.split()
          if not any(c.isdigit() for c in w)] for t, _ in corpus]
cbow = Word2Vec(sents, vector_size=60, window=5, min_count=2, sg=0,
                epochs=40, seed=42, workers=1)
skip = Word2Vec(sents, vector_size=60, window=5, min_count=2, sg=1,
                negative=10, epochs=40, seed=42, workers=1)
print(f"\n5. WORD2VEC : vocabulary = {len(skip.wv)} words, 60 dimensions")
for w in ["contact", "mobile", "pan"]:
    if w in skip.wv:
        nn = ", ".join(f"{a}({b:.2f})" for a, b in skip.wv.most_similar(w, topn=4))
        print(f"   skip-gram nearest to {w:<9}: {nn}")
for a, b in [("contact", "mobile"), ("contact", "pan")]:
    if a in skip.wv and b in skip.wv:
        print(f"   similarity({a}, {b}) : CBOW = {cbow.wv.similarity(a, b):.3f}, "
              f"skip-gram = {skip.wv.similarity(a, b):.3f}")
```

Output

```
1. CORPUS : 800 sentences
   sample : Employee Rohan Mehta of Symbiosis Institute has PAN HFTRX0402A.

2. CONTEXT WINDOWS : 7341 total, 1376 PII / 4129 non-PII in train
   example (PII) : registered mobile is <TARGET> .
```

```
3. VECTORIZATION
   BoW    matrix (5505, 395)  vocabulary = 395
   TF-IDF matrix (5505, 395)  w(t,d) = tf(t,d) x log(N / df(t))
   lowest  IDF (common)     : <target>, ., the, pan, at
   highest IDF (informative): wbkjt3026b., wdzkd9879h., xosjk1688b.
```

```
4. PII-CONTEXT CLASSIFICATION
   Features + model           Accuracy   Prec    Recall   F1
   BoW    + MultinomialNB     0.714      0.476   0.994    0.644
   TF-IDF + MultinomialNB     0.794      0.577   0.787    0.665
   BoW    + LogisticReg       0.996      1.000   0.983    0.992
   TF-IDF + LogisticReg       0.954      0.926   0.893    0.909
   context words most indicative of PII :
      with, of, employee, works, please, registered, pan, applicant
```

```
5. WORD2VEC : vocabulary = 85 words, 60 dimensions
   skip-gram nearest to contact  : today(0.93), on(0.93), or(0.92), please(0.86)
   skip-gram nearest to mobile   : raised(0.96), registered(0.95), by(0.95), ticket(0.95)
   skip-gram nearest to pan      : employee(0.79), has(0.74), of(0.74), submitted(0.73)
   similarity(contact, mobile) : CBOW = 0.399, skip-gram = 0.433
   similarity(contact, pan) : CBOW = 0.282, skip-gram = 0.326
```

Explanation of the Output

The document-term matrix has 5505 rows and only 395 columns, and almost every entry is zero, which illustrates the sparsity of count-based representations. The inverse document frequency behaves as expected: the placeholder <TARGET>, the full stop and the word the receive the lowest weights because they occur in nearly every window, while the rarest terms are individual identifier strings.

Logistic Regression clearly outperforms Naive Bayes on both representations, reaching an F1 of 0.992 with Bag of Words against 0.644 for Naive Bayes. The gap is explained by the independence assumption: the words in a context window are strongly correlated, which violates that assumption but not the assumptions of Logistic Regression. The scores are high in absolute terms because the corpus is

generated from a small set of templates, so the contexts are more regular than real text would be; this is a limitation of synthetic data and is reported rather than concealed.

The Naive Bayes ranking places employee, registered, pan, applicant and please among the context words most indicative of PII. Several of these are the trigger words that were chosen by hand for the confidence model in Assignment 2, so a rule written manually there is partly recovered from data here.

In the word2vec space the identifier-announcing vocabulary groups together, with mobile closest to registered, ticket and raised, and pan closest to employee and submitted. Skip-gram gives a higher similarity than CBOW for both trigger pairs, which is consistent with skip-gram being the better choice for relatively infrequent words. The nearest neighbours are nevertheless driven partly by template co-occurrence rather than true semantics, since the vocabulary is only 85 words.

Conclusion

In this experiment text was converted into numerical features and dense word vectors. One-hot encoding, Bag of Words and TF-IDF were compared, and the inverse document frequency was shown to suppress common function words while preserving informative terms. Context windows of three tokens on each side were extracted with the target token masked, so that the classifiers learned only from the surrounding words and never from the entity string itself. Logistic Regression reached an F1 of 0.992 against 0.644 for Naive Bayes on the same Bag of Words features, which is explained by the conditional independence assumption being violated by correlated context words. The context words the classifier ranked highest coincided in part with the trigger words chosen by hand for the confidence model of Assignment 2, and word2vec placed that same identifier-announcing vocabulary close together in the embedding space, so two independent methods support the context-boosting rule. Skip-gram produced higher similarities than CBOW for the trigger pairs, as expected for relatively infrequent words. The embeddings produced here are used to initialise the embedding layer of the deep learning model built in the next assignment.